

第 43 讲：Regularization 与 Weight Decay

为什么给模型增加约束，反而可能让它在新数据上表现更好

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【科研洞察】

第 41 讲已经看到：模型可以把 Training Loss 压得很低，却在 Validation / Test 上越来越差。第 42 讲又建立了严格的数据边界。现在的问题是：**如果模型太容易把训练集记住，我们能不能在训练过程中主动加入“约束”，让它少追随训练集里的偶然噪声？** Regularization（正则化）就是在回答这个问题。

一、这一讲到底要解决什么问题？

假设你有一个容量很大的模型，它面对训练数据时非常灵活：

- 真正稳定的规律，它能学；
- 训练集里的随机噪声，它也能学；
- 某几个偶然出现的异常样本，它甚至也能记住。

如果我们只看 Training Loss，就可能得到一个错觉：

$$L_{train} \downarrow \Rightarrow \text{模型越来越好}$$

但真正关心的是：

$$L_{test} \downarrow \quad \text{或未来真实任务表现提升}$$

【原理】

Regularization（正则化）的核心思想不是“让训练更难”本身，而是**限制模型利用某些过于自由、过于复杂或不稳定的拟合方式**，希望把学习压力推向更稳定、可泛化的规律。

二、通俗解释：为什么“限制模型”反而可能更好？

可以把模型想成一名学生。

如果考试前给他一本包含答案的题库，而且允许无限记忆，那么他可以把每一道题的答案都背下来。此时：

- 题库原题：100 分；
- 换一套题：可能很差。

正则化相当于增加一些限制，例如：

- 不能把每道题都单独死记；

- 尽量使用更简单、更平滑的规则；
- 不要让少数参数变得异常巨大；
- 训练时随机丢掉一部分神经元，迫使网络不要依赖单一路径；
- Validation 变差时及时停止，不要继续把噪声学进去。

这些限制会让 Training Loss 可能不再是最低，但未见数据上的误差反而可能更低。

三、Regularization 不是一种单独技术，而是一整类思想

常见正则化方法包括：

方法	它在限制什么	本课程位置
Weight Decay / L2	参数不能无限变大	本讲重点
Dropout	不允许网络长期依赖固定神经元子集	第 44 讲
Early Stopping	不允许无限继续拟合训练集噪声	已在第 41 讲引入
Data Augmentation	扩大“同一规律的不同表现形式”	以后按需
Label Smoothing	不让分类目标极端过度自信	以后按需
模型容量控制	限制层数、宽度、参数量	本讲会讨论

【常见误区 / 风险】

Regularization 不是“保证泛化”的魔法。约束太弱可能没有效果；约束太强又会让模型连真正规律都学不够，产生 Underfitting（欠拟合）。

四、从普通 Training Objective 到带正则的 Objective

普通监督学习最基本的目标是：

$$J(\theta) = L_{data}(\theta)$$

也就是只要求模型把数据拟合好。

L2 Regularization 会把目标改成：

$$J(\theta) = L_{data}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

这里第一次出现两个“目标”：

- L_{data} ：你原本真正关心的预测错误；
- $\frac{\lambda}{2} \|\theta\|_2^2$ ：对大参数的惩罚。

公式逐项拆解

$$\theta = [\theta_1, \theta_2, \dots, \theta_k]$$

则：

$$\|\theta\|_2^2 = \theta_1^2 + \theta_2^2 + \dots + \theta_k^2$$

所以新的目标可以读成：

“预测要准，但参数也不要为了追训练集噪声而无限膨胀。”

五、一个具体数字例子：Regularization 到底加了什么？

假设：

$$w = [3, 4]$$

则：

$$\|w\|^2 = 3^2 + 4^2 = 25$$

如果当前 Data Loss：

$$L_{data} = 2$$

并且：

$$\lambda = 0.1$$

那么：

$$\frac{\lambda}{2} \|w\|^2 = \frac{0.1}{2} \times 25 = 1.25$$

新的 Training Objective：

$$J = 2 + 1.25 = 3.25$$

【具体例子】

模型现在不再只问“怎样让 Data Loss 最小”，还会问“为了再降低一点 Data Loss，我是不是必须付出一个很大的参数范数代价？”

六、为什么 L2 会推动参数变小？——直接看 Gradient

对：

$$\frac{\lambda}{2} \|\theta\|^2$$

求导：

$$\nabla_{\theta} \left(\frac{\lambda}{2} \|\theta\|^2 \right) = \lambda \theta$$

因此：

$$\nabla J = \nabla L_{data} + \lambda \theta$$

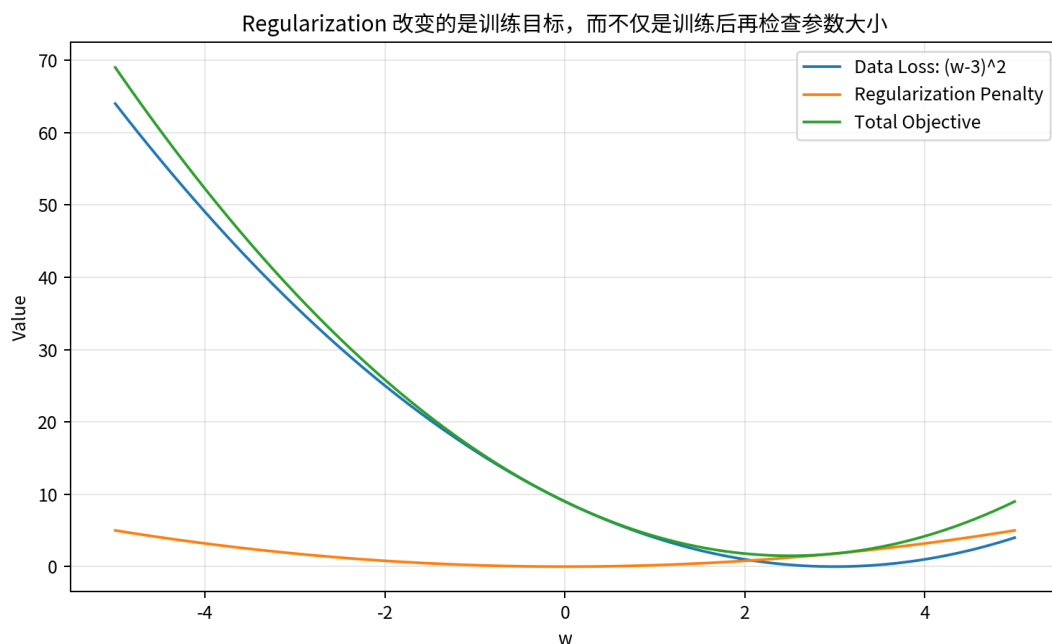


图 1: Regularization 改变的是训练 Objective: Data Loss 与 Parameter Penalty 同时参与。

普通 SGD 更新:

$$\theta_{t+1} = \theta_t - \eta(\nabla L_{data} + \lambda\theta_t)$$

整理:

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\nabla L_{data}$$

这就是第 39 讲见过的 “Shrinkage (收缩)” 直觉。

【原理】

对最基础的 SGD, L2 Regularization 与这种 Weight Decay 形式可以非常直接地建立代数对应; 但对 Adam 等自适应优化器, 不能把这个等价关系机械照搬, 所以现代 Transformer / LLM 常使用 AdamW 做 decoupled weight decay。

七、矩阵参数时, L2 Penalty 到底怎么算?

实际神经网络的 Parameter 通常不是一个标量, 而是矩阵甚至更高维 Tensor。

例如一个 Linear Layer:

$$W = \begin{bmatrix} 1 & -2 & 0.5 \\ 0 & 1.5 & -1 \end{bmatrix}$$

Shape:

$$W.shape = (2, 3)$$

L2 penalty 并不关心 “第几行第几列”, 而是把所有元素平方后求和:

$$\|W\|_F^2 = 1^2 + (-2)^2 + 0.5^2 + 0^2 + 1.5^2 + (-1)^2 = 8.5$$

这里严格说是 Frobenius norm 的平方，但在深度学习代码里常统一理解成 “Parameter Tensor 所有元素的平方和”。

对应梯度仍然具有完全相同 Shape:

$$\frac{\partial}{\partial W} \left(\frac{\lambda}{2} \|W\|_F^2 \right) = \lambda W$$

所以:

$$W.shape = (2, 3) \implies (\lambda W).shape = (2, 3)$$

这与前面学过的 “Parameter Gradient 与 Parameter 自身 Shape 对应” 完全一致。

八、手算一次带 Weight Decay 的 SGD Update

取一个最简单的标量参数:

$$w = 4$$

当前 Data Gradient:

$$g_{data} = 3$$

Learning Rate:

$$\eta = 0.1$$

Weight Decay:

$$\lambda = 0.2$$

L2/SGD 对应的总梯度:

$$g_{total} = g_{data} + \lambda w = 3 + 0.2 \times 4 = 3.8$$

因此:

$$w_{new} = 4 - 0.1 \times 3.8 = 3.62$$

也可以拆开理解:

$$(1 - \eta\lambda)w = 0.98 \times 4 = 3.92$$

再做 Data Gradient Update:

$$3.92 - 0.1 \times 3 = 3.62$$

【具体例子】

同一次更新里实际上有两股力量: Data Gradient 试图让预测更准; Weight Decay 试图把 Parameter 往 0 的方向收缩。最终更新是两者共同作用的结果。

九、一个很容易踩坑的代码细节: `loss.item()` 不一定包含 Weight Decay

假设:

```

1 loss = mse(model(x), y)
2 loss.backward()
3 optimizer.step()
4 print(loss.item())

```

如果使用：

```

1 optimizer = torch.optim.SGD(
2     model.parameters(),
3     lr=0.02,
4     weight_decay=0.1,
5 )

```

那么 `loss.item()` 通常仍然只是你显式计算的 MSE，而不是：

$$MSE + \frac{\lambda}{2} \|\theta\|^2$$

Weight Decay 是 Optimizer 在更新阶段额外应用的规则。

这意味着训练日志中可能同时存在两个值得区分的量：

- **Data Loss**：模型对数据拟合得怎样；
- **Regularized Objective**：理论上带 penalty 的优化目标。

【常见误区 / 风险】

看到两个实验的 `train loss` 数值一样，并不代表它们的 Parameter Update 一样。如果 Optimizer 的 `weight decay` 不同，即使同一个 `data loss`、同一个 `data gradient`，实际 Parameter Update 仍可能不同。

十、Learning Rate 与 Weight Decay 为什么会耦合？

从基础 SGD：

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta g_t$$

可以直接看到每一步收缩因子：

$$1 - \eta\lambda$$

所以同样：

$$\lambda = 0.1$$

如果：

$$\eta = 10^{-3}$$

每一步收缩很弱；如果：

$$\eta = 10^{-1}$$

每一步收缩明显得多。

训练很多步后，累计效果近似与：

$$\prod_t (1 - \eta_t \lambda)$$

有关。

因此现实训练里：

Weight Decay 强度不能完全脱离 Learning Rate Schedule 单独解释。

Warmup、Cosine Decay、Training Steps 改变后，即使配置文件仍写同一个 `weight_decay=0.1`，累计收缩轨迹也可能不同。

十一、L1 与 L2：为什么这一课重点放 L2 / Weight Decay ?

L1 Regularization:

$$J = L_{data} + \lambda \sum_i |\theta_i|$$

L2 Regularization:

$$J = L_{data} + \frac{\lambda}{2} \sum_i \theta_i^2$$

最粗略的区别：

- L1 更容易产生很多接近或等于 0 的参数，常与 sparsity 联系；
- L2 更像连续地收缩所有参数，是现代神经网络中更常见的基础正则直觉；
- Transformer / LLM 优化配置里常见的是 AdamW Weight Decay，而不是直接对所有参数做 L1。

当前课程目标不是系统研究稀疏优化，因此只需要知道 L1 存在，以及 “Regularization 不只一种 penalty” 即可。

十二、显式 Regularization 与隐式 Regularization

显式 Regularization 是你明确写进目标函数或训练规则的约束，例如：

- L2 / Weight Decay；
- Dropout；
- Label Smoothing。

而深度学习里还有一些**隐式 Regularization** 来源，例如：

- Mini-Batch SGD 的随机性；
- Early Stopping；
- Optimization Dynamics 对某类解的偏好；
- 数据增强改变了模型能够轻易记忆的模式。

这里不需要深入理论，只需要建立一个科研意识：

【科研洞察】

一个模型的 Generalization 不是只由 `weight_decay` 一个旋钮决定。Optimizer、Batch Size、Training Duration、Augmentation、Model Architecture 等都会共同形成最终 inductive bias。

十三、Weight Decay 真正在“惩罚什么”？

Weight Decay 并不是直接说：

“某个 Weight 很大，所以这个 Feature 一定错了。”

更准确的理解是：它改变了模型可接受的解的偏好。

如果两个参数配置在 Training Data 上都差不多好：

$$L_{data}(\theta_A) \approx L_{data}(\theta_B)$$

但：

$$\|\theta_A\| \ll \|\theta_B\|$$

那么有正则时会更偏向 θ_A 。

在很多问题中，这种偏好会减少模型对小样本噪声和偶然方向的过度追随。

【常见误区 / 风险】

不要把“参数小”机械理解成“模型简单”或“泛化一定好”。神经网络存在参数缩放、重参数化、Normalization 等复杂性。Weight Decay 是一种有用的 inductive bias (归纳偏置)，不是普适真理。

十四、实验主线：100 个 Feature，只有 8 个真的有用

为了把 Generalization 看清楚，本讲实验专门构造：

- Feature dimension: $D = 100$;
- Train samples: 120;
- Validation samples: 1000;
- Test samples: 1000;
- 真正有用 Feature: 前 8 个;
- 剩余 92 个 Feature: 纯噪声;
- 目标中额外加入随机噪声。

模型本身只是一个：

$$\hat{y} = Xw$$

这个实验非常适合观察：模型会不会利用大量噪声 Feature 去追训练集中的偶然波动。

十五、实验结果 1: Training 最低，不代表 Test 最好

Weight Decay	Train MSE	Val MSE	Test MSE	$\ w\ _2$
0	0.0475	1.9324	2.0636	3.4990
0.01	0.0495	1.5472	1.6492	3.3750
0.03	0.0591	1.2484	1.3223	3.2229
0.10	0.1080	1.1466	1.1897	2.9582
0.30	0.2872	1.5611	1.5713	2.6121
1.00	1.0500	2.8201	2.7907	2.0571

最关键的现象是：

$$wd = 0$$

时 Training MSE 最低，但 Test 很差。

增加到：

$$wd = 0.1$$

Training MSE 反而变差，但 Test 明显变好。

继续增加到：

$$wd = 1.0$$

则 Train / Validation / Test 全部变差，说明正则化太强已经造成 Underfitting。

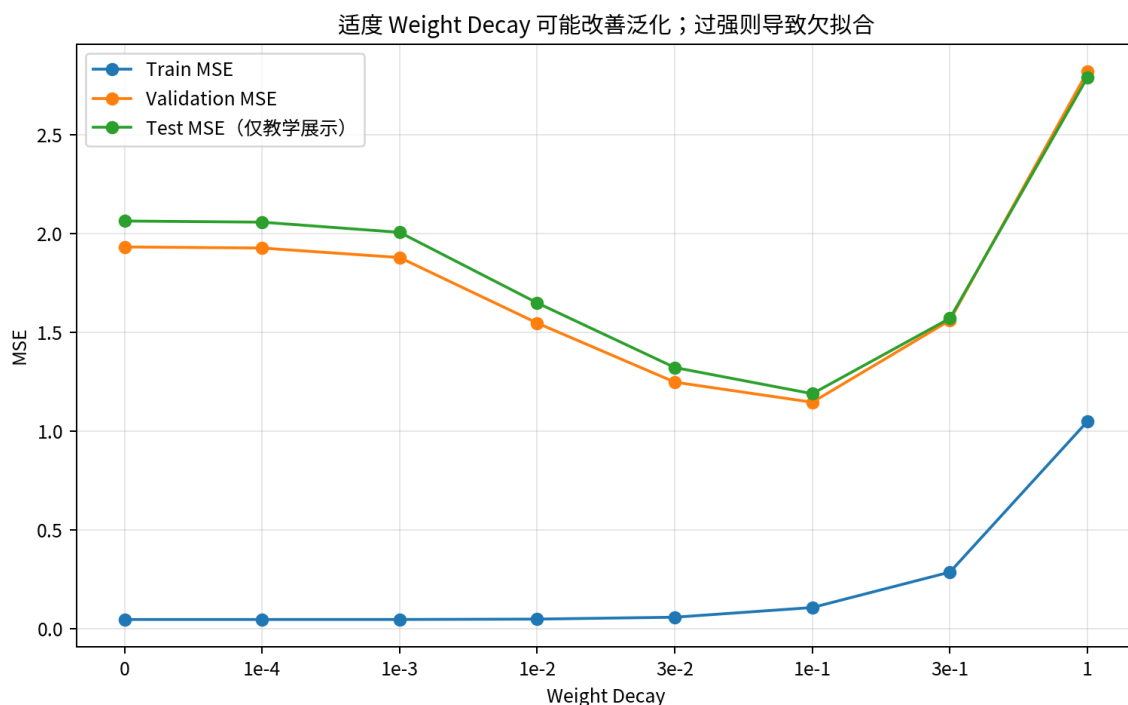


图 2: Regularization 的典型 U 型逻辑：太弱容易过拟合，适度改善泛化，太强造成欠拟合。

十六、为什么适度 Weight Decay 会改善这个实验？

这个任务里 100 个 Feature 中只有 8 个真正与目标有关。

没有正则时，模型可以利用剩余 92 个 Feature 的偶然相关性，把 Training Error 继续压低。由于 Train 只有 120 个样本，这些偶然相关在独立数据上很难重复出现。

实验中：

- $wd=0$ 时噪声 Feature 平均 $|w|$ 约 0.1073；
- $wd=0.1$ 时降到约 0.0715；
- 同时 learned weight 与 true weight 的 cosine similarity 从 0.9272 提高到 0.9539。

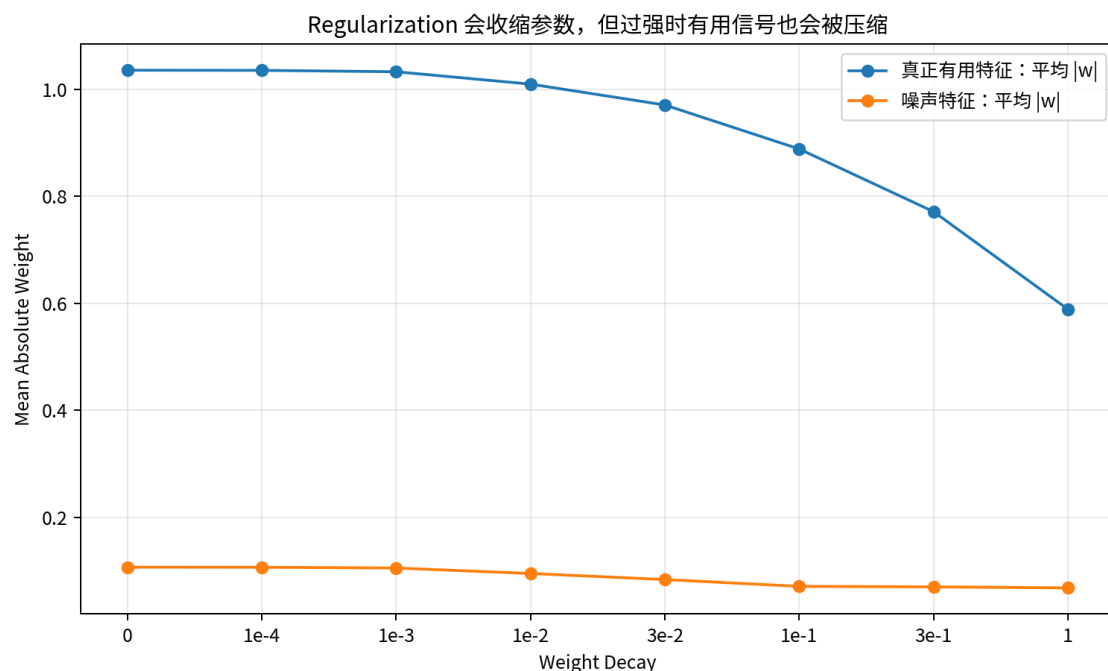


图 3: Weight Decay 会收缩参数；在本实验中，噪声方向被明显压缩，同时真正有用方向仍保留较多信号。

十七、为什么 Weight Decay 过强又会失败？

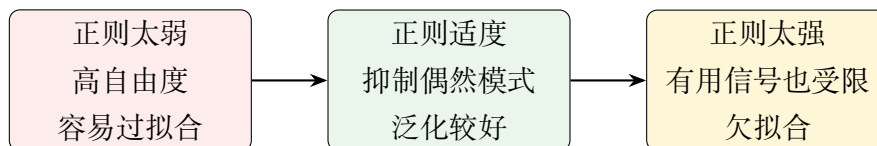
正则化不是“只压噪声，不碰信号”的智能筛选器。

当 λ 太大时，它同样会压缩真正有用的 Weight。

实验中：

- $wd=0.1$ 时真正有用 Feature 平均 $|w| \approx 0.8883$ ；
- $wd=1.0$ 时已经压到约 0.5890；
- Test MSE 也从 1.1897 恶化到 2.7907。

这就是典型的 Bias-Variance Trade-off 直觉：



十八、Parameter Norm 与 Generalization: 能看，但不能机械下结论

实验里随着 Weight Decay 增大:

$$\|w\|_2 : 3.499 \rightarrow 2.057$$

整体不断下降。

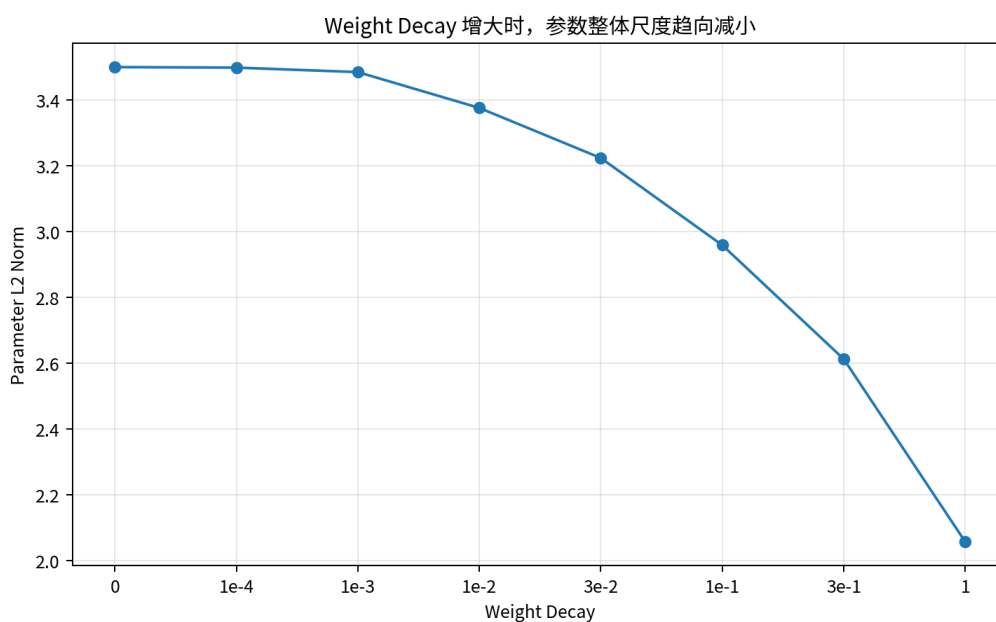


图 4: Weight Decay 越强，参数整体 L2 Norm 趋向更小。

但是 Validation/Test 并不是单调变好:

$$\text{Test MSE} : 2.064 \rightarrow 1.190 \rightarrow 2.791$$

所以正确结论是:

【科研洞察】

Parameter Norm 是 Regularization 行为的一个诊断量，但不能把“Norm 越小”机械等同于“泛化越好”。真正的超参数选择仍然必须依赖严格 Validation，并由独立 Test 最终确认。

十九、Regularization、Optimization、Data: 三者必须分开

训练结果不好时，不要只会加 Weight Decay。

问题来源	常见现象	首先检查什么
Optimization	Train Loss 本身就降不下去	LR、Gradient、Initialization、Optimizer
Overfitting	Train 很好，Val/Test 明显差	Regularization、数据量、模型容量
Data Leakage	Val/Test 异常漂亮	Split unit、preprocessing、duplicates
Distribution Shift	IID Val 好，真实域差	Evaluation domain、robustness、adaptation

【常见误区 / 风险】

如果 Training Loss 都很高，第一反应通常不应该是“再加 Regularization”。那可能是在给一个本来就没学会的模型继续加限制。

二十、与 Dropout / Early Stopping 的关系

三者目标相似，但机制不同：

- **Weight Decay**：直接偏好较小参数；
- **Dropout**：训练时随机删除部分激活路径，减少对固定共适应模式的依赖；
- **Early Stopping**：当 Validation 不再改善时停止，限制继续拟合噪声的时间。

它们不是互斥的。现实模型可能同时使用：

AdamW + Dropout + Early Stopping

但每个方法都可能与其他超参数发生交互，因此 Ablation 不能只看单变量名字。

二十一、PyTorch 中 Weight Decay 写在哪里？

最基础 SGD：

```
1 optimizer = torch.optim.SGD(  
2     model.parameters(),  
3     lr=0.02,  
4     weight_decay=0.1,  
5 )
```

现代 Transformer / LLM 中更常见：

```
1 optimizer = torch.optim.AdamW(  
2     model.parameters(),  
3     lr=3e-4,  
4     weight_decay=0.1,  
5 )
```

但是两段代码虽然都写 `weight_decay`，底层机制不能不加区分地混为一谈——这正是第 39 讲已经学过的内容。

二十二、Parameter Groups: 为什么有些参数常常不做 Weight Decay ?

实际 Transformer 训练常见做法是把参数分组:

- 大型 Weight Matrix: 通常 decay;
- Bias: 常设 no-decay;
- LayerNorm / RMSNorm 的 scale: 也常设 no-decay。

示意代码:

```
1 decay_params = []
2 no_decay_params = []
3
4 for name, p in model.named_parameters():
5     if name.endswith("bias") or "norm" in name.lower():
6         no_decay_params.append(p)
7     else:
8         decay_params.append(p)
9
10 optimizer = torch.optim.AdamW([
11     {"params": decay_params, "weight_decay": 0.1},
12     {"params": no_decay_params, "weight_decay": 0.0},
13 ], lr=3e-4)
```

【常见误区 / 风险】

这是一种常见工程实践, 不是“所有模型都必须如此”的自然定律。看论文或代码时应记录具体 parameter-group 规则。

二十三、与 Transformer / LLM 的直接关系

你以前在 LLM 课程里见过 AdamW, 现在可以从 Generalization 角度重新理解它。

LLM 训练中常见配置会报告:

- Optimizer = AdamW;
- Learning Rate;
- Weight Decay;
- Warmup / Scheduler;
- Dropout;
- Global Batch / Tokens;
- Gradient Clipping。

这些不是彼此孤立的超参数。Weight Decay 的实际效果会和：

LR Schedule, Optimizer State, Training Duration, Parameter Groups

共同决定最终参数轨迹。

二十四、与 Agent Systems 科研的关系

Agent Systems 里，Regularization 未必只表现为“给一个小网络加 L2”。你更常遇到的是微调、Adapter / LoRA、Reward Model、Policy Model 或小型辅助模块。

这时需要注意：

- Train tasks 很少，但模型容量很大，特别容易记住 task-specific pattern；
- 相同 trajectory family 重复出现，会与第 42 讲的数据泄漏问题叠加；
- Weight Decay 改善某个 benchmark，不代表跨 task family / environment 也改善；
- 如果做 Ablation，必须固定 Seed、数据划分、Optimizer、Steps、LR Schedule；
- 报告 Train/Val/Test，而不是只报告最终 benchmark 分数。

【科研洞察】

以后做 Agent Systems 实验，如果某个训练技巧让 Validation 提升，第一问题不应只是“分数更高了吗？”，还应该问：它到底是在减少过拟合、改变优化速度、改变参数尺度，还是仅仅与某个特定 split / task family 更匹配？

二十五、科研中怎样公平比较 Weight Decay？

一个最低限度的 Ablation 应尽量控制：

1. 相同 Dataset 与 Split；
2. 相同模型初始化或多 Seed；
3. 相同 Optimizer 类型；
4. 相同 Learning Rate / Schedule；
5. 相同 Optimizer Steps / Tokens Seen；
6. 只改变 Weight Decay；
7. 用 Validation 选超参数；
8. Test 最后一次确认；
9. 同时记录 Train Loss、Val/Test Metric、Parameter Norm；
10. 必要时记录 no-decay parameter group。

只比较：

$$wd = 0 \quad vs \quad wd = 0.1$$

然后看到 0.1 更好，并不能证明 0.1 是“普适最优”。本讲实验本身就显示：更强的 1.0 已经明显过度正则化。

二十六、常见误区总结

【常见误区 / 风险】

1. **误区：** Train Loss 更高，所以 Weight Decay 一定有害。
纠正： Regularization 的目标本来就不是把 Training Loss 压到最低，而是改善未见数据表现。
2. **误区：** Weight 越小，模型泛化一定越好。
纠正： 过强收缩会造成 Underfitting。
3. **误区：** L2、Adam weight decay、AdamW 完全一样。
纠正： 第 39 讲已经说明，自适应 Optimizer 中必须区分 coupled 与 decoupled。
4. **误区：** 加了 Weight Decay 就不会过拟合。
纠正： 它只是一个 inductive bias，仍需 Validation / Test 验证。
5. **误区：** Validation 最好的 weight decay 可以直接拿 Test 再反复微调。
纠正： 那会破坏第 42 讲建立的评估边界。

二十七、本讲最低闭环能力

学完以后，你至少应该能回答：

1. Regularization 想解决什么问题？
2. 为什么 Training Loss 变差，Validation/Test 反而可能变好？
3. L2 Objective 中 $\lambda \|\theta\|^2 / 2$ 每一项是什么意思？
4. 为什么最基础 SGD 下它会产生参数收缩？
5. Weight Decay 太弱、适度、太强分别可能出现什么？
6. 为什么 Parameter Norm 只能做诊断，不能直接等同 Generalization？
7. AdamW 和普通 Adam 的 Weight Decay 为什么不能混为一谈？
8. 一篇论文的 Weight Decay Ablation 至少应该控制哪些变量？

【本节小结】

本讲主线：**Regularization** 不是让模型 “训练得更漂亮”，而是在训练目标和训练过程里加入偏好，牺牲一部分对 **Training Data** 的自由拟合能力，换取更好的 **Generalization**。Weight Decay 是最常见的一种参数正则化，但它必须通过严格 Validation/Test 来确定强度，太弱和太强都可能失败。